

Ottodot Take-Home Planning Package

Status: planning only - no implementation has been created.

This package translates the Ottodot Full-Stack Engineer take-home brief into an implementation-ready plan. The source brief remains authoritative if anything here conflicts with it.

Recommended direction

Build a deliberately small, backend-led trial-booking slice. Use a relational database and make booking confirmation a single atomic transaction. The confirmation transaction locks the selected class, re-checks confirmed occupancy, verifies successful payment, and then confirms at most one booking for the final seat. Database constraints provide a second line of defense against duplicate active bookings.

Suggested concrete stack for the later implementation:

- TypeScript
- Next.js with a minimal App Router UI and route handlers
- PostgreSQL
- Drizzle ORM plus explicit SQL where locking or partial indexes are clearer
- Vitest for unit/integration tests and Playwright only if time remains
- Docker Compose for one-command database startup

The stack is a recommendation, not a requirement. The concurrency model matters more than the framework.

Development and architecture posture

- Use compact spec-driven development: approve the requirements, design, acceptance criteria, and ordered tasks before implementation.
- Implement reliability with a PostgreSQL transaction and an explicit booking state machine.
- Do not implement a distributed Saga for the take-home. Document Saga-like compensation as the production evolution for real payment, void, and refund operations.
- Do not introduce Kafka without demonstrated needs for multiple independent consumers, durable replay, or event throughput. A transactional outbox is the first production step if reliable publication becomes necessary.
- Use one short-lived feature branch and clear commits. Full GitFlow is deliberately avoided because this exercise has no release train or parallel long-lived development streams.

These decisions are positive evidence of scope control. Saga and Kafka are included as considered alternatives, not presented as implemented components.

Documents

File	Purpose
01-analysis-and-feedback.md	Interpretation, risks, feedback, and scope advice
02-product-requirements.md	Product scope, actors, requirements, acceptance criteria, and assumptions
03-technical-design.md	Architecture, schema, APIs, invariants, concurrency, errors, and observability
04-technical-decisions.md	Concise architecture decision records and tradeoffs
05-flowcharts.md	Mermaid system, sequence, state, and decision diagrams
06-test-strategy.md	Verification matrix, concurrency test design, seed data, and quality gates
07-implementation-prompt.md	Structured prompt for a later coding session
08-submission-plan.md	Four-hour plan, README/AI usage outlines, demo script, and final checklist

Core invariants

1. A class has no more than `capacity` confirmed bookings; the seeded capacity is 4.
2. A student has at most one active booking for a given class.
3. Only a booking with a recorded successful payment result may become confirmed.
4. Failed payment never places a student on the confirmed roster.
5. Roster output contains confirmed bookings only.
6. Repeated commands and payment callbacks are idempotent.

Important product caveat

"Payment succeeded" and "seat acquired" are separate facts. In the take-home mock, a successful payment result can arrive after another user has taken the last seat. That booking must not be confirmed. It should transition to `capacity_unavailable`, retain an auditable payment attempt, and return a clear message. A production design should authorize first and capture only after the seat is claimed, or automatically void/refund the payment.